Network Troubleshooting Expert System

A simple guide: from the database to the finished project

React | Spring Boot | PostgreSQL

What is an expert system?

1

Facts

Answers from the user, like internet_available = false.

2

Knowledge base

Rules saved as data in the database, not in the code.

3

Inference engine

Java code that checks the facts against the rules.

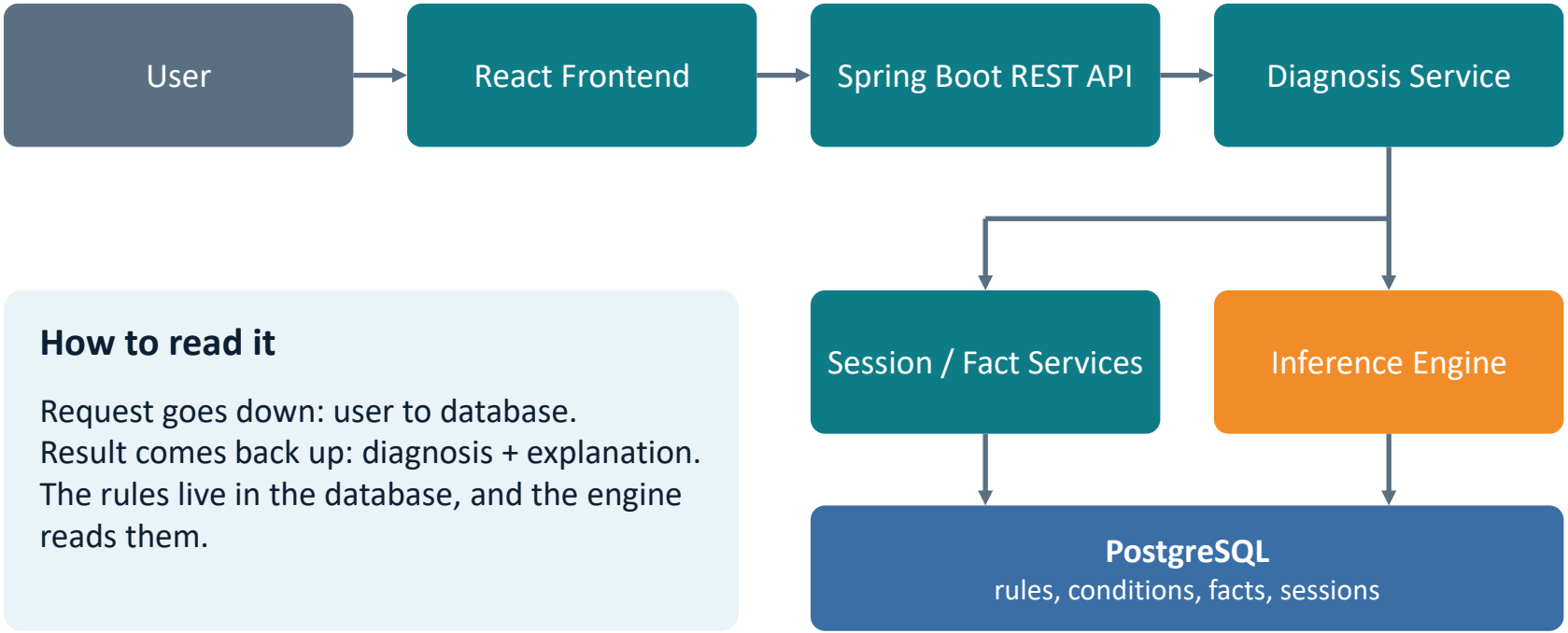
4

Explanation

Shows which rules fired, so the user knows why.

Facts + Rules = Diagnosis + Explanation

The whole system in one graph



How to read it

Request goes down: user to database.
Result comes back up: diagnosis + explanation.
The rules live in the database, and the engine reads them.

Roadmap: from database to finish

1 Database

Create the 7 tables with schema.sql.

Started

2 Backend setup

Spring Boot project, entities, repositories.

Planned

3 Knowledge base

Add real rules and conditions (seed data).

Planned

4 Engine + API

Inference engine and REST endpoints.

Planned

5 Frontend

React screens: questions and result.

Planned

6 Test and finish

Tests, security, ERD and documentation.

Planned

Part 1: The database (7 tables)

Knowledge base (the rules)

rules
Rule name, priority, enabled or not

rule_conditions
What must be true for the rule

rule_conclusions
What the rule says as a result

Session data (each user run)

users
Account details

diagnosis_sessions
One troubleshooting session + final diagnosis

session_facts
Answers the user gave in a session

matched_rules
Which rules fired in a session

How one rule looks as data

Example rule: "Router problem" (priority 10)

fact name	operator	expected value
wifi_connected	=	true
internet_available	=	false
router_reachable	=	false

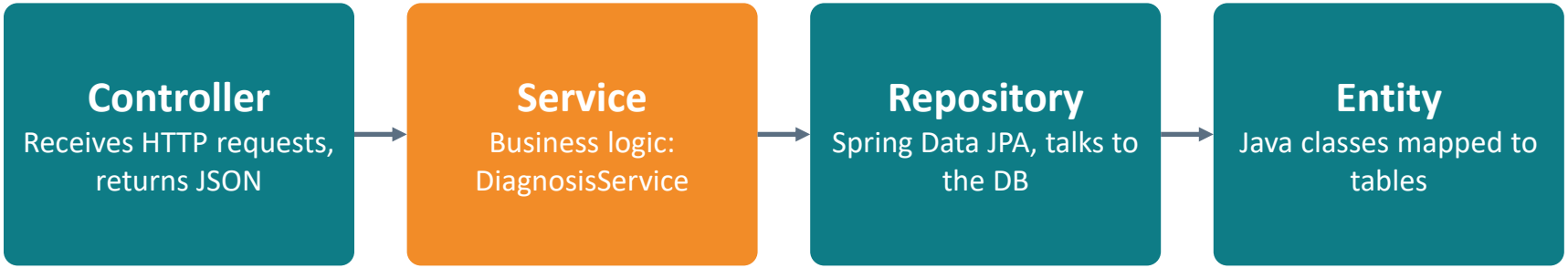
Rule fires only if ALL conditions are true.
Extra facts that the rule does not need are ignored.

Example only. Your real column names are in schema.sql.

Conclusion
Possible router problem. Restart the router and check the cables.

Facts from the user
wifi_connected = true
internet_available = false
router_reachable = false
cable_connected = true (not used)

Part 2: The Spring Boot backend



Suggested structure (not fixed by the README):
The inference engine is its own class, separate from the controllers.
Use DTO classes for JSON, so entities stay inside the backend.
Set spring.jpa.hibernate.ddl-auto=validate: Spring checks your schema, it does not change it.

Part 3: The inference engine

1. **Load facts**
   Read the answers from session_facts.

2. **Load rules**
   Only enabled rules, sorted by priority.

3. **Check conditions**
   A rule matches if every condition is true.

4. **Save matches**
   Write them to matched_rules.

5. **Choose result**
   Best matching rule gives the diagnosis.

```
facts = loadFacts(session)
rules = loadEnabledRules()

for rule in rules:
  if allConditionsMatch(rule, facts):
    saveMatchedRule(rule)
    results.add(rule.conclusion)

diagnosis = best(results)
return diagnosis + explanation
```

Part 4: The REST API

order	method	endpoint	purpose
1	POST	/api/diagnosis-sessions	Create a session
2	POST	/api/diagnosis-sessions/{id}/facts	Send the answers
3	POST	/api/diagnosis-sessions/{id}/diagnose	Run the engine
4	GET	/api/diagnosis-sessions/{id}	Get session details
5	GET	/api/diagnosis-sessions/{id}/explanation	Get the explanation
-	GET	/api/rules	List all rules

These endpoints are planned. They are not available until you build the controllers and services.

Part 5: The React frontend

1

Start

Begin a new troubleshooting session.

2

Questions

One simple question at a time, like Yes / No.

3

Result

Show the possible diagnosis and what to try.

4

Explanation

Show which rules matched and why.

React only shows the screens. The diagnostic rules stay in the database.

One diagnosis, step by step

1	**User answers**	Wi-Fi connected: yes. Internet: no. Router reachable: no.
2	**Create session**	POST /api/diagnosis-sessions saves a new session.
3	**Send facts**	POST .../facts saves the answers in session_facts.
4	**Run engine**	POST .../diagnose loads the rules and checks them.
5	**Save matches**	The "Router problem" rule fires and goes to matched_rules.
6	**Show result**	The user sees the diagnosis and the explanation.

Part 6: Test it and protect it

Testing

- Unit tests for the inference engine
- Integration tests for the REST API
- Check the full React flow by hand
- Test rules with known facts

Security

- Never commit passwords or tokens
- Use environment variables
- Use a restricted database user
- Validate all API input
- Hash passwords if you add users

Where to start now

1. Run database/schema.sql in PostgreSQL
2. Create the Spring Boot project and entities
3. Add 5 to 10 real rules as seed data
4. Build the inference engine with unit tests
5. Add the API, then the React screens

Later: ERD, architecture diagram, authentication, license file.